

FleetFlow — Documentação do sistema

Documentação técnica do FleetFlow, a operação de pátios de veículos que atende **Stellantis** (ativação e desmobilização de frota em Salvador, Recife e Natal) e **Unidas** (guarda de veículos em Salvador).

⚠ **Este repositório é público.** Nenhum documento aqui contém chaves de API, tokens de acesso, senhas ou hashes. Segredos ficam nas variáveis de ambiente da Vercel, nos secrets do Supabase e no gerenciador de credenciais — nunca no Git. Ao editar estas páginas, mantenha essa regra.

Última revisão: **14/08/2026**.

Índice

#	Documento	O que cobre
01	Visão geral	O que o sistema faz, quem usa, quais são as peças
02	Arquitetura	Diagrama, componentes, domínios, deploy, fluxo de dados
03	Banco de dados	Schema, tabelas, funções, cron, RLS, storage, legado
04	Regras de negócio	Matriz de cobrança, diárias, contratos, regras de "não fazer"
05	PWA dos operadores	<code>app.fleetflow.digital</code> — fluxos, telas, gravação no banco
06	Dashboard e assistente de IA	<code>dash.fleetflow.digital</code> — medição, KPIs, chat SQL
07	Integrações	Vexsoft, portais Unidas/Refran, push, bot de Telegram
08	Operação e runbook	Deploy, monitoramento, tarefas comuns, troubleshooting
09	Histórico e conciliações	Linha do tempo e os fechamentos que corrigiram a base
10	Pendências e riscos	Dívida técnica, achados de segurança, o que está em aberto

Leitura mínima

Quem vai **mexer no código** deve ler 02, 03, 04 e o documento do componente que vai tocar. Quem vai **operar / fechar medição** deve ler 04, 06 e 08. Quem está **retomando o projeto do zero** deve ler tudo, na ordem.

Convenções

- **Praças:** `SSA` (Salvador/BA), `REC` (Recife/PE), `NAT` (Natal/RN).
- **Fuso:** toda regra de negócio usa `America/Bahia` (UTC−3). O banco guarda em UTC.
- **Moeda:** valores em R\$ aparecem sempre como `cliente / parceiro` — o que a Stellantis paga e o que o pátio terceirizado recebe.
- **"Cliente"** = quem paga o FleetFlow (Stellantis, Unidas). **"Parceiro"/"fornecedor"** = quem opera o pátio (AZPark, Via Total, Refran).

01 — Visão geral

O que o FleetFlow faz

O FleetFlow controla a **entrada, permanência e saída de veículos em pátios terceirizados** e transforma esses movimentos em **cobrança para o cliente** e **custo para o parceiro**. Dois contratos convivem no mesmo sistema, com modelos de cobrança completamente diferentes:

Stellantis — serviço de pátio (SSA, REC, NAT). Cada veículo é cobrado individualmente: uma taxa no momento da entrada (que varia conforme o carro esteja em *ativação* ou *desmobilização*) mais uma diária de custódia por dia de permanência, com sete dias de carência para o cliente. A fonte de verdade é a tabela `vehicle_service_charges` — uma linha por cobrança.

Unidas — guarda de veículos (só SSA). Não há cobrança por veículo. A conta é feita sobre a **ocupação diária agregada** do pátio: um piso mensal que cobre 50 carros/dia e um valor por carro-dia excedente. A fonte de verdade é a tabela `daily_occupancy` — um snapshot por dia, pátio e cliente.

Em Salvador, o pátio físico é alugado da **Refran**, que cobra pela ocupação total (Stellantis + Unidas somados). A diferença entre o que a Unidas paga e o que a Refran cobra é a **margem da guarda**.

Quem usa

Perfil	Quem	Onde
Operador de pátio	Bira (AZPark) — SSA Diego (Via Total) — REC Marcos (Via Total RN) — NAT	PWA <code>app.fleetflow.digital</code>
Administração	Chico	Dashboard <code>dash.fleetflow.digital</code> + SQL/assistente
Cliente Unidas	Equipe Unidas	Portal <code>unidas.fleetflow.digital</code>
Fornecedor Refran	Equipe Refran	Portal <code>refran.fleetflow.digital</code>

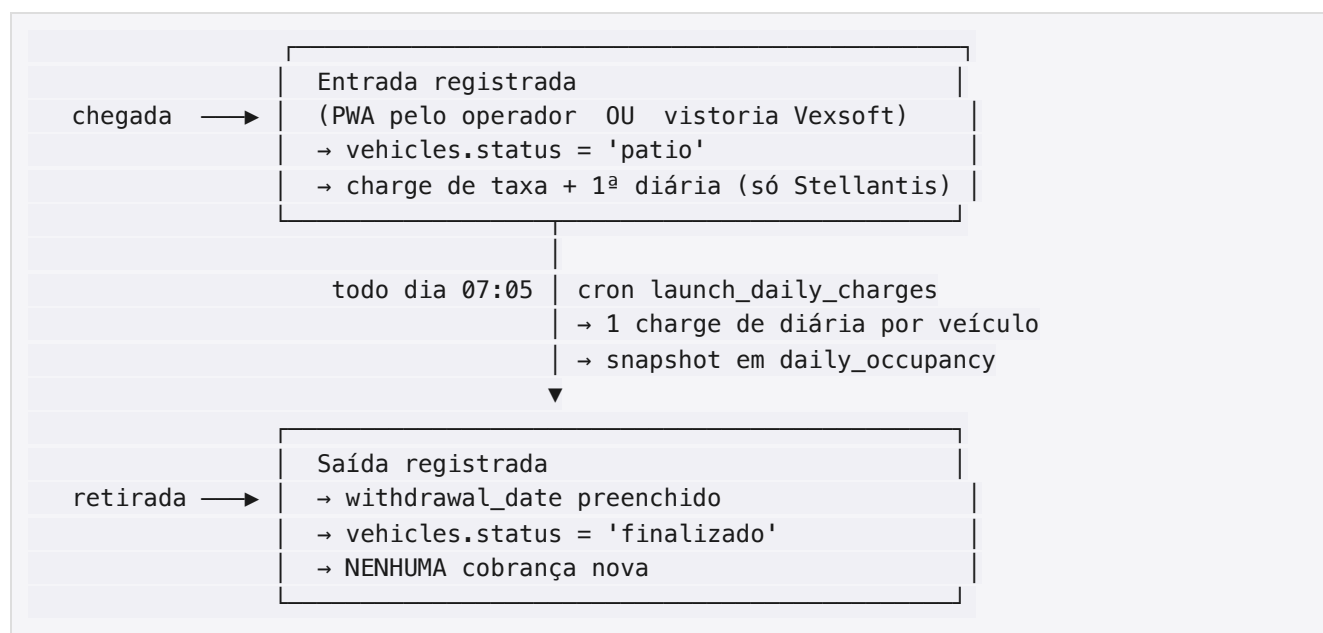
Os operadores são prestadores terceirizados. **Nenhuma tela mostrada a eles exibe valores em R\$** — nem o que o cliente paga, nem o que o parceiro recebe. Ver [regras de negócio](#).

As peças do sistema

Peça	O que é	Onde roda
Pátio PWA	App instalável dos operadores. Foto → OCR de placa por IA → confirmação → gravação	Vercel (Next.js 14)
Banco	Postgres com todo o estado: veículos, cobranças, ocupação, contratos	Supabase (<code>sa-east-1</code>)
Dashboard interno	KPIs, medição mensal com export, seção Unidas × Refran, assistente de IA	GitHub Pages (HTML único)

Peça	O que é	Onde roda
Assistente de IA	Chat que responde perguntas e monta relatórios consultando o banco em modo leitura	Rota <code>/api/ask</code> dentro do PWA
Pipeline Vexsoft	Robô horário que lê vistorias no Gmail, abre o PDF e lança o movimento	Python no Mac mini (launchd)
Portais externos	Duas páginas estáticas de prestação de contas, uma para a Unidas e outra para a Refran	Vercel + Edge Functions
Cron de diárias	Job diário que lança as diárias e recalcula os snapshots de ocupação	<code>pg_cron</code> no Supabase

O ciclo de vida de um veículo



Duas coisas que costumam confundir quem chega agora:

"Ativação" e "desmobilização" descrevem o carro, não o movimento. Ativação é um 0 km entrando na frota; desmobilização é um usado saindo dela. Os dois passam pelo pátio e os dois têm uma entrada e uma saída. O que define a taxa é o *tipo do carro*, cobrado no momento da *entrada*.

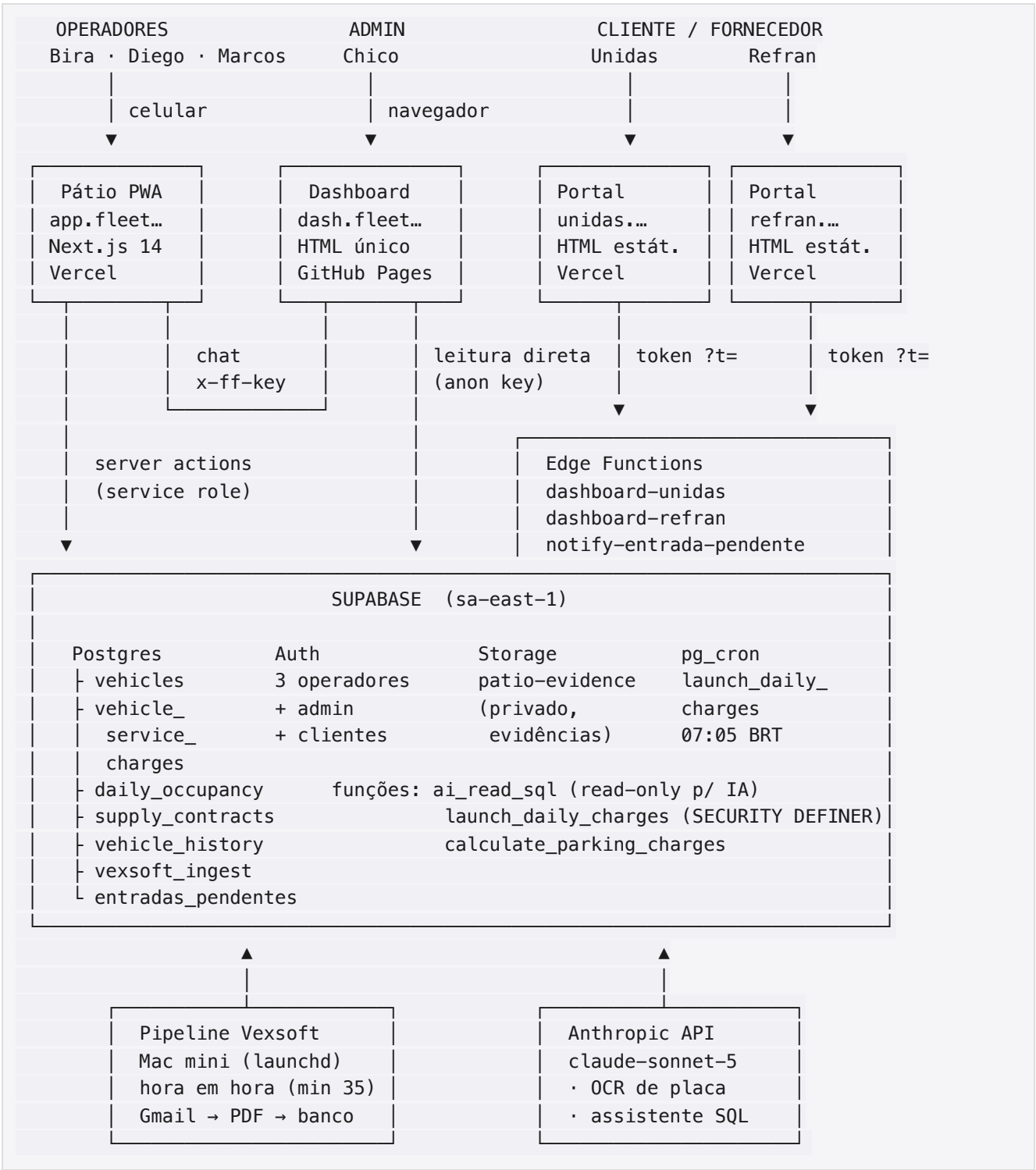
A saída nunca gera cobrança. Ela só fecha o ciclo. Quem cobra a permanência é o cron, dia a dia, enquanto o veículo estiver com `status = 'patio'`.

Números da operação (14/08/2026)

- 188 veículos cadastrados — 42 em pátio, 145 finalizados, 1 arquivado
- 3.527 cobranças lançadas — R\$ 42.170 de receita bruta acumulada contra R\$ 29.630 de custo de parceiro
- Distribuição por pátio: SSA 112, REC 70, NAT 6
- Medição fechada abril-julho/2026: **R\$ 27.070**
- Contratos Unidas e Refran vigentes desde 27/07/2026

02 — Arquitetura

Visão em uma tela



Componentes

Pátio PWA — app.fleetflow.digital

Next.js 14 (App Router) + TypeScript + Tailwind, hospedado na Vercel (projeto `patio-pwa`). É o único componente com backend próprio: usa **server actions** e uma **route handler** (`/api/ask`), e é onde ficam as duas chaves sensíveis do sistema (`ANTHROPIC_API_KEY` e `SUPABASE_SERVICE_ROLE_KEY`). Também serve os portais estáticos em `/unidas` e `/refran` por *rewrite*.

O código-fonte vive **apenas** em `~/Desktop/Fleet Flow/patio-pwa/` e na Vercel — **não é um repositório Git**. Isso é um risco conhecido, registrado em [pendências](#).

Detalhes em [05 — PWA dos operadores](#).

Banco — Supabase `sqbjxabftqtzdigjhivl`

Projeto `fleetflow-teste` (o nome é herança; **é produção**), região `sa-east-1`. Concentra Postgres, Auth, Storage, Edge Functions e o agendador `pg_cron`. Convive no mesmo projeto com o schema `fitwellness` (outro produto) e com tabelas herdadas do projeto **ProLine**, que hoje estão vazias.

Detalhes em [03 — Banco de dados](#).

Dashboard interno — `dash.fleetflow.digital`

Um único `index.html` servido pelo GitHub Pages a partir do repositório `chicocbrandao/fleetflow-dashboard`. Carrega Chart.js, SheetJS, jsPDF e o SDK do Supabase por CDN. Lê o banco **diretamente com a chave anônima** (as tabelas relevantes têm policy de leitura para `anon`) e calcula a medição no navegador. O acesso é protegido por uma senha comparada por hash no próprio JavaScript.

Detalhes em [06 — Dashboard e assistente](#).

Assistente de IA — `/api/ask`

Rota dentro do PWA que recebe uma pergunta em linguagem natural do dashboard, conversa com o Claude em modo *tool use* e devolve resposta, tabela e gráfico. As consultas passam pela função `ai_read_sql`, que executa sob um papel Postgres **sem nenhum privilégio de escrita**.

Portais externos

Duas páginas HTML estáticas, uma por contraparte, com projeto Vercel próprio e também publicadas dentro do PWA (`/unidas`, `/refran`). Não falam com o banco: chamam Edge Functions dedicadas que aplicam o recorte de dados de cada uma. O acesso é por **token na query string**.

Pipeline Vexsoft

Script Python no **Mac mini** (launchd, hora em hora no minuto 35) que lê os e-mails de vistoria da Vexsoft no Gmail (IMAP), abre o PDF, identifica placa/tipo/pátio e lança o movimento no banco. Desde 13/08/2026 é a **entrada principal de dados da Stellantis**; o PWA virou backup para esse cliente. A Unidas continua 100% no PWA.

Detalhes em [07 — Integrações](#).

Domínios e DNS

Zona `fleetflow.digital`, gerenciada no **Hostinger**.

Host	Aponta para	Servido por
<code>app.fleetflow.digital</code>	projeto <code>patio-pwa</code>	Vercel
<code>dash.fleetflow.digital</code>	<code>chicocbrandao.github.io</code>	GitHub Pages
<code>unidas.fleetflow.digital</code>	projeto <code>fleetflow-unidas</code>	Vercel
<code>refran.fleetflow.digital</code>	projeto <code>fleetflow-refran</code>	Vercel

`fleetflow.com.br` não está sob nosso controle. O registro está em nome da Semplice, com nameservers na KingHost (site e e-mail). Sem acesso ao painel, não é possível criar subdomínios nesse domínio.

Uma pegadinha registrada: o arquivo `CNAME` no repositório **não** foi suficiente para o GitHub Pages assumir o domínio customizado — foi preciso forçar via API (`gh api -X PUT repos/.../pages -f cname=...`).

Onde vive cada segredo

Segredo	Onde está	Quem usa
<code>ANTHROPIC_API_KEY</code>	Env var da Vercel (projeto <code>patio-pwa</code>)	OCR de placa e assistente
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Env var da Vercel	Assistente (<code>/api/ask</code>)
Chave anônima do Supabase	Embutida em HTML público (dashboard e portais)	Leitura pelas policies de <code>anon</code>
Senha do dashboard	Só na cabeça do Chico; o hash está no HTML	Login do dash e do assistente
Tokens dos portais	Constantes dentro das Edge Functions	Acesso Unidas/Refran
Chaves VAPID (push)	Constantes dentro da Edge Function de push	Notificação de pendência

O padrão "constante dentro do código" para tokens e VAPID é dívida conhecida — ver [10 — Pendências e riscos](#).

Fluxo de dados: de onde vem cada número

Número	Origem	Calculado onde
Medição mensal Stellantis	<code>vehicle_service_charges</code>	Dashboard (JS) e assistente (SQL)
Fatura Unidas	<code>daily_occupancy</code> + <code>supply_contracts</code>	Edge Function <code>dashboard-unidas</code> e dashboard
Repassse Refran	<code>daily_occupancy</code> (todos os clientes) + <code>supply_contracts</code>	Edge Function <code>dashboard-refran</code> e dashboard
Carros em pátio "hoje"	<code>vehicles</code> com <code>status='patio'</code> — ao vivo	Dashboard

Número	Origem	Calculado onde
Ocupação histórica	daily_occupancy — snapshot das 07:05	Cron

A distinção da última linha importa: o snapshot **não enxerga** lançamentos feitos depois das 07:05 do dia. Por isso o cron recalcula uma **janela deslizante de 7 dias** a cada execução, corrigindo automaticamente lançamentos retroativos recentes. Além de 7 dias o histórico congela e correções precisam ser feitas à mão.

03 — Banco de dados

Projeto Supabase `sqbjxabftqtzdigjhivl` (`fleetflow-teste` — é produção), região `sa-east-1`.
Extensões relevantes: `pg_cron 1.6.4`, `pg_net 0.20.0`, `pgcrypto`, `pg_stat_statements`.

O schema `public` tem 33 tabelas, mas **apenas 10 fazem parte do FleetFlow**. O resto é herança do projeto ProLine (ver [legado](#)). Há ainda um schema `fitwellness`, de outro produto, que não tem relação com o pátio.

As tabelas que importam

`vehicles` — um registro por ciclo de veículo

188 linhas. É o centro do modelo.

Coluna	Tipo	Observação
<code>id</code>	uuid PK	
<code>client_id</code>	uuid NOT NULL	FK → <code>profiles(id)</code> . Stellantis ou Unidas
<code>plate</code>	varchar NOT NULL	UNIQUE global — ver limitação abaixo
<code>model</code> , <code>brand</code>	text NOT NULL	
<code>year</code>	int NOT NULL	o PWA grava o ano corrente quando não sabe
<code>chassi</code>	varchar NOT NULL UNIQUE	placeholder <code>PENDING-{placa}</code> quando não há dado
<code>status</code>	text	<code>patio</code> · <code>finalizado</code> · <code>archived</code> (texto livre, sem CHECK; default herdado é <code>'active'</code>)
<code>estimated_arrival_date</code>	date	data de entrada no pátio
<code>withdrawal_date</code>	timestampz	data de saída
<code>yard</code>	text	<code>SSA</code> · <code>REC</code> · <code>NAT</code> . Existe desde 24/07/2026
<code>notes</code>	text	trilha textual do movimento (ver formato abaixo)
<i>(demais)</i>		<code>color</code> , <code>renavam</code> , <code>km</code> , <code>fuel_type</code> , <code>photos</code> , <code>preparacao</code> , <code>comercializacao</code> ... herdadas do ProLine, pouco usadas

Índices: só PK, `plate` e `chassi`. Não há índice em `client_id`, `yard` nem `status`.

Formato de `notes`: `{TIPO} {MOVIMENTO} – {PRAÇA} ({Operador} {Empresa})`, por exemplo `ATIVACAO ENTRADA – SSA (Bira AZPark)`. Na entrada o campo é **sobrescrito**; na saída o texto novo é **concatenado** com `|`.

Como descobrir o tipo do carro a partir de `notes`: vale a palavra `ATIVACAO / DESMOBILIZACAO` que aparecer **primeiro**, porque o primeiro segmento é sempre a entrada — e é a entrada que define a taxa

cobrada. ENTRADA / SAIDA são o eixo do movimento e só servem de fallback para registros legados.

Duas limitações do modelo que já causaram problema:

- 1. plate é UNIQUE global, então o banco não representa a reentrada do mesmo carro. Um veículo que passa duas vezes pelo pátio não pode ter dois registros. Caso real: TVH8D65 / TYH8D65.
- 2. withdrawal_date é timestamptz gravado à meia-noite UTC. Aplicar at time zone 'America/Sao_Paulo' volta um dia (mostra 21:00 do dia anterior). O valor cru é o correto — use withdrawal_date::date sem conversão.

vehicle_service_charges — a fonte de verdade financeira

3.527 linhas. Uma linha por cobrança.

Coluna	Tipo	Observação
vehicle_id	uuid NOT NULL	FK → vehicles
service_key	text NOT NULL	ativacao · desmobilizacao · diaria · remocao_plotagem · reset_luz_painel · troca_vidro_triangular
client_charge	numeric NOT NULL	o que a Stellantis paga
partner_cost	numeric NOT NULL	o que o pátio recebe
margin	numeric	coluna gerada: client_charge - partner_cost
charge_date	date	dia a que a cobrança se refere
entry_date	date	data de entrada do veículo (nas diárias)
days_count	int	sempre 1 no modelo atual
grace_applied	bool	true quando o dia caiu na carência
status	text	pending (2.363) · cancelled (1.164)
pricing_id	uuid	FK → service_pricing
notes	text	rastro do lançamento

Regra de ouro: todo somatório de faturamento precisa filtrar status <> 'cancelled'. Um terço das linhas está cancelada — resultado das conciliações de julho e agosto.

Índice único uniq_charge_vehicle_service_date sobre (vehicle_id, service_key, charge_date), aplicado apenas às linhas com status <> 'cancelled' — cancelar uma cobrança libera o slot, que é o que faz o botão Desfazer e os relançamentos continuarem funcionando.

Ele existe desde 14/08/2026. Antes disso a idempotência dependia só do NOT EXISTS dentro de launch_daily_charges(), e três linhas duplicadas passaram: uma dupla submissão do PWA em REC (TEY8C68, 09/06, R\$ 88 cobrados duas vezes) e uma diária de entrada repetida em SSA (QMZ7C56, 22/05). Foram canceladas na mesma data.

Distribuição: `diaria` 3.455, `desmobilizacao` 45, `ativacao` 23, extras 4. Período coberto: 01/04/2026 a 14/08/2026.

`daily_occupancy` — **snapshot de ocupação**

111 linhas. Uma linha por (`dia`, `pátio`, `cliente`), com `UNIQUE (occ_date, yard, client_id)` — o alvo do `ON CONFLICT` do cron.

Coluna	Observação
<code>occ_date</code>	dia do snapshot
<code>yard</code>	pátio
<code>client_id</code>	Stellantis ou Unidas
<code>car_count</code>	veículos naquele dia
<code>computed_at</code>	quando o cron calculou

É a base **exclusiva** da cobrança Unidas e do repasse Refran. Cobre de 01/07/2026 em diante.

`supply_contracts` — **contratos de ocupação**

2 linhas. Parametriza os cálculos de ocupação sem hardcode.

<code>code</code>	<code>kind</code>	<code>contraparte</code>	<code>pátio</code>	<code>inclui</code>	<code>mensal</code>	<code>excedente</code>	<code>início</code>
<code>unidas_ssa</code>	<code>client</code>	Unidas	SSA	50 carros	R\$ 9.000	R\$ 6,00/carro-dia	27/07/2026
<code>refran_ssa</code>	<code>supplier</code>	Refran	SSA	50 vagas	R\$ 3.500	R\$ 4,00/carro-dia	27/07/2026

`service_pricing` — **tabela de preços**

7 linhas. **Nunca hardcode preço**: leia daqui.

<code>service_key</code>	<code>cliente</code>	<code>parceiro</code>	<code>carência cliente</code>	<code>extra?</code>
<code>ativacao</code>	190,00	123,50	0	não
<code>desmobilizacao</code>	88,00	57,20	0	não
<code>diaria</code>	10,00	6,50	7 dias	não
<code>diaria_unidas</code>	6,00	0,00	0	não
<code>remocao_plotagem</code>	350,00	50,00	0	sim
<code>reset_luz_painel</code>	250,00	100,00	0	sim
<code>troca_vidro_triangular</code>	2.500,00	1.500,00	0	sim

`UNIQUE (service_key, client_id, partner_id)` permite preço global (com os dois nulos) e preço específico por cliente — é assim que `diaria_unidas` coexiste com `diaria`.

`vehicle_history` — **auditoria de movimentação**

99 linhas. `action` segue o padrão `{tipo}_{movimento}`: `ativacao_entrada`, `desmobilizacao_saida`, `guarda_entrada`, `guarda_saida`, mais `undo` para lançamentos desfeitos. `performed_by` guarda o `auth.uid()` do operador (sem FK).

`vexsoft_ingest` — log da ingestão automática

Uma linha por e-mail de vistoria processado. `gmail_message_id` é UNIQUE — é o que garante idempotência do pipeline. O campo `acao` registra a decisão tomada: `entrada_criada`, `saida_lancada`, `duplicado_pwa` ou `excecao`.

`entradas_pendentes` — fila de exceção

Alimentada pelo pipeline quando uma **saída** é detectada por vistoria mas o veículo nunca teve entrada registrada. O operador resolve pela tela de pendências do PWA informando a data de entrada. Tem um trigger `AFTER INSERT` que dispara push notification.

`profiles` — usuários

12 linhas, espelha `auth.users`. Enum `user_role`: `client`, `specialist`, `supplier`, `admin`, `finance`, `operator`. A coluna `yard` (com CHECK em `SSA|REC|NAT`) é o que amarra cada operador ao seu pátio.

Contém também os dois "clientes": **Stellantis** (`167748aa-2fee-49df-9b0f-140919bfbf7c`) e **Unidas** (`fea28c45-8ca5-4550-98ad-3f9e78b7a120`) — esses UUIDs aparecem hardcoded em vários pontos do código.

`push_subscriptions`

Assinaturas Web Push dos operadores. **Hoje está vazia**, o que significa que a notificação de pendência não chega a ninguém — ver [pendências](#).

Funções

Função	Segurança	O que faz
<code>launch_daily_charges()</code>	DEFINER	Job diário: lança diárias e recalcula snapshots
<code>ai_read_sql(q text)</code>	INVOKER	Gateway SQL somente-leitura do assistente
<code>calculate_parking_charges(...)</code>	INVOKER	Cálculo avulso de diárias de um veículo (pouco usado)
<code>notify_entrada_pendente()</code>	DEFINER	Trigger → chama a Edge Function de push

Função	Segurança	O que faz
<code>get_users_count</code> , <code>get_pending_users</code> , <code>get_clients_with_vehicle_count</code> , <code>get_unread_notifications_count</code> , <code>get_pending_checklist_reviews[_count]</code>	DEFINER	Legado ProLine , não usadas pelo pátio

launch_daily_charges() — o coração da cobrança

Roda em `America/Bahia`. Três etapas:

1. Lê o preço de `diaria` em `service_pricing`. Se não achar, aborta com `{"ok": false}`.
2. **Insere uma diária por veículo**, mas **só para a Stellantis** (o `client_id` está hardcoded na função). Critério: `status = 'patio'` e `estimated_arrival_date <= hoje`. Um `NOT EXISTS` por `(vehicle_id, service_key='diaria', charge_date)` evita duplicar. Cobra R\$ 0 do cliente durante a carência de 7 dias e o preço cheio depois; o custo do parceiro é lançado desde o dia 1.
3. **Recalcula os snapshots de ocupação numa janela de 7 dias** (`hoje-6` até `hoje`), com `INSERT ... ON CONFLICT DO UPDATE`, e em seguida **deleta** as linhas de `daily_occupancy` daquele dia que não têm mais veículo correspondente — porque um `upsert` sozinho nunca zeraria uma contagem.

A janela de 7 dias é o que faz lançamentos retroativos recentes se autocorrigirem. Retroativos com mais de 7 dias exigem recálculo manual.

ai_read_sql(q text) — o gateway do assistente

Barreiras, em ordem:

1. Rejeita qualquer consulta que não comece com `SELECT` ou `WITH`.
2. Rejeita mais de um comando (qualquer `;` interno).
3. `SET LOCAL ROLE ff_ai_readonly` — **a garantia real**: um papel sem nenhum privilégio de escrita, sem `USAGE` em `auth`, `storage`, `cron` ou `vault`, e sem `BYPASSRLS`.
4. `SET LOCAL statement_timeout = '20s'`.
5. Envelopa a consulta em `select * from (<q>) _q limit 2000` — o que, de quebra, faz o Postgres recusar CTEs que modificam dados ("*data-modifying statement must be at the top level*").
6. `RESET ROLE` no sucesso e no erro.

O `EXECUTE` foi revogado de `anon` e `authenticated`; só `service_role` chama.

Agendamento

Um único job de `pg_cron`:

jobid	nome	schedule	comando
2	launch-daily-charges	5 10 * * * (UTC) = 07:05 America/Bahia	<code>SELECT</code> <code>public.launch_daily_charges();</code>

Histórico contínuo e sem falhas, duração típica de 0,15 a 0,35 s.

Documentos antigos citam 03:05 UTC. **O valor correto é 5 10 * * *** — o horário foi mudado em julho/2026 para casar com a regra "todo dia às 7h" dos contratos de ocupação.

Edge Functions

Função	Criada	O que faz
<code>dashboard-unidas</code>	24/07/2026	Alimenta o portal da Unidas: contrato, carros no pátio agora, série diária do mês, excedentes e fatura parcial
<code>dashboard-refran</code>	24/07/2026	Alimenta o portal da Refran: ocupação total do pátio SSA (todos os clientes) e repasse. Não expõe placas, clientes nem receita
<code>notify-entrada-pendente</code>	13/08/2026	Recebe o POST do trigger e dispara Web Push para todas as assinaturas; remove assinaturas mortas (404/410)

Todas com `verify_jwt: true` — a chamada precisa levar a chave anônima no header, e o controle de acesso real é o token na query string.

Storage

Bucket	Público	Objetos	Uso
<code>patio-evidence</code>	não	128 (71 MB)	Fotos das movimentações. Path <code>{yard}/{yyyy-mm-dd}/{uuid}.jpg</code>
<code>evidencias</code>	sim	3 (14 MB)	Legado ProLine
<code>medicoes-tmp</code>	sim	0	Temporário de medição

Policies do `patio-evidence`: o operador só consegue subir na pasta do próprio pátio (validado contra `profiles.yard`); a leitura é liberada para qualquer autenticado, porque a server action precisa baixar a imagem para mandar ao modelo de visão.

RLS

RLS está **habilitado em todas as 33 tabelas**. Três padrões convivem:

- **ProLine:** `Admin full access` (tudo, para `role='admin'`) + leitura livre para `authenticated`.
- **Pátio:** policies específicas por papel — `operator` lê e insere veículos, `operator|admin` atualiza, `operator|admin` insere cobranças. Não há policy de UPDATE nem DELETE em `vehicle_service_charges` (correções passam por SQL administrativo).
- **Assistente:** `ff_ai_readonly_select` em toda tabela, apenas SELECT, apenas para o papel `ff_ai_readonly`.

Quatro tabelas são legíveis pela chave anônima, sem login: `vehicles` (incluindo placa e chassi), `daily_occupancy`, `supply_contracts` (valores de contrato) e `vexsoft_ingest`. É o que permite o dashboard funcionar como HTML estático — e é também um risco documentado em [10](#).

Papéis

O único papel customizado é `ff_ai_readonly` : sem login, sem `BYPASSRLS` , com `USAGE` apenas em `public` e `SELECT` nas tabelas. Tudo o mais é padrão do Supabase.

Tabelas legadas

Vazias e sem uso pelo pátio, todas herdadas do **ProLine** — um produto anterior de gestão de serviços automotivos que usou o mesmo banco:

`addresses` , `audit_logs` , `collection_requests` , `invoices` , `mechanics_checklist` , `mechanics_checklist_items` , `mechanics_checklist_evidences` , `notifications` , `part_requests` , `partner_services` , `partners_service_categories` , `parts_inventory` , `payment_transactions` , `quote_admin_approvals` , `quote_items` , `reports` , `service_pricing_history` , `services` , `supplier_fees` , `supplier_price_tables` .

Duas ainda têm dados residuais: `service_orders` (65) e `quotes` (55). E `clients` / `partners` **não são tabelas** — são views sobre `profiles` filtrando por `role` .

Migrations aplicadas

Versão	Nome
20250311120000	<code>init_schema</code> (base ProLine)
20260520180717	<code>allow_anon_vehicle_insert_update</code>
20260525145050	<code>add_operator_role_and_yard_column</code>
20260525145103	<code>add_yard_column_to_profiles</code>
20260525155832	<code>enable_rls_operator_policies</code>
20260525155935	<code>create_desmobilize_vehicle_rpc</code>
20260525161022	<code>harden_desmobilize_vehicle_search_path</code>
20260525161147	<code>remove_bot_neo_anon_policies</code>
20260525185240	<code>drop_recursive_admin_profiles_policy</code>
20260525193942	<code>drop_old_desmobilize_rpc</code>
20260525194005	<code>create_launch_daily_charges_and_cron</code>
20260525230425	<code>vehicle_history_insert_policy</code>
20260611113208	<code>fitwellness_init</code>
20260724144446	<code>unidas_refran_phase1</code>
20260729015116	<code>occupancy_trailing_window</code>
20260813142035	<code>vexsoft_ingest_pipeline</code>
20260814151155	<code>entradas_pendentes_push</code>
20260814174103	<code>ai_readonly_role_and_policies</code>
20260814174127	<code>ai_read_sql_function</code>
20260814174148	<code>ai_read_sql_function_v2</code>

Versão	Nome
20260814·	harden_definer_function_grants
20260814·	unique_charge_per_vehicle_service_date

04 — Regras de negócio

Este documento é a autoridade sobre cobrança. Ele **substitui** o `FleetFlow_Regras_Medicao.pdf` original (maio/2026), que tratava "ativação = entrada" e "desmobilização = saída" como sinônimos — uma simplificação que se mostrou errada.

O modelo de duas dimensões

Toda movimentação tem **dois campos independentes**:

Campo	Valores	Significa
<code>tipo_carro</code>	<code>ativacao</code> <code>desmobilizacao</code>	a natureza do veículo: 0 km entrando na frota, ou usado saindo dela
<code>movimento</code>	<code>entrada</code> <code>saida</code>	o que está acontecendo agora: o carro está chegando ou indo embora

São quatro combinações possíveis. Confundi-las é a causa histórica número 1 de erro de faturamento.

Matriz de cobrança — Stellantis

Tipo do carro	Movimento	O que é cobrado
Ativação	Entrada	Taxa de ativação R\$ 190 / R\$ 123,50 + 1ª diária
Ativação	Saída	Nada. Só fecha o ciclo
Desmobilização	Entrada	Taxa de desmobilização R\$ 88 / R\$ 57,20 + 1ª diária
Desmobilização	Saída	Nada. Só fecha o ciclo

O padrão: **a entrada cobra a taxa do tipo mais a primeira diária; a saída não cobra nada**. As diárias intermediárias vêm do cron.

A taxa é definida pelo tipo da ENTRADA e não muda depois. Se um carro entra como ativação e sai classificado como outra coisa, a taxa de R\$ 190 já cobrada permanece. Decisão registrada em 14/08/2026 (caso TDZ4D72, que saiu como "Venda Seminovo").

"Venda Seminovo" — tipo de operação que passou a aparecer nos PDFs da Vexsoft — deve ser cobrado **como desmobilização**.

Diárias

	Cliente (Stellantis)	Parceiro (pátio)
Valor	R\$ 10,00/dia	R\$ 6,50/dia
Carência	7 dias — dias 1 a 7 saem R\$ 0	nenhuma — cobra desde o dia 1

- A **primeira diária** é lançada junto com a taxa, no momento da entrada.
- As **demais** são lançadas pelo cron `launch_daily_charges`, todo dia às **07:05 (America/Bahia)**, uma por veículo com `status = 'patio'`.
- **O dia de entrada e o dia de saída contam.**
- O cron é idempotente: não duplica a diária de um dia já lançado.

Extras

Nunca aparecem no aplicativo do operador. **Só o admin lança**, por SQL, quando é avisado.

Serviço	Cliente	Parceiro
<code>remocao_plotagem</code>	R\$ 350,00	R\$ 50,00
<code>reset_luz_painel</code>	R\$ 250,00	R\$ 100,00
<code>troca_vidro_triangular</code>	R\$ 2.500,00	R\$ 1.500,00

Antes de lançar um extra, **sempre confira que a placa existe** — ver [o que nunca fazer](#).

Contrato Unidas — guarda em Salvador

Modelo completamente diferente: **não gera** `vehicle_service_charges`. A conta sai da ocupação diária.

- R\$ 6,00 por carro-dia
- Piso mensal de **R\$ 9.000**, equivalente a 50 carros/dia
- Vigência desde **27/07/2026**

$$\text{fatura_do_mês} = 9.000 + \sum \text{dias} \quad \text{máx}(0, \text{carros_unidas_no_dia} - 50) \times 6$$

No PWA o fluxo Unidas é **só entrada e saída**, sem escolher tipo de carro e sem gerar nenhuma cobrança. O seletor de cliente só aparece para o operador de **SSA** — o contrato é exclusivo de Salvador, e o servidor recusa um lançamento Unidas vindo de outro pátio.

Contrato Refran — custo do pátio em Salvador

- 50 vagas por **R\$ 3.500/mês**
- Excedente a **R\$ 4,00 por carro-dia**
- **A ocupação conta Stellantis + Unidas somados**
- Vigência desde **27/07/2026**

Margem da guarda

$$\text{margem} = \text{fatura_Unidas} - \text{repass_Refran}$$

Com os dois lados dentro dos respectivos limites, o piso é **R\$ 9.000 – R\$ 3.500 = R\$ 5.500/mês**. Acima disso, cada carro-dia excedente rende R\$ 6 de receita contra R\$ 4 de custo — um *spread* de R\$ 2.

O detalhe que muda tudo: **os gatilhos de excedente são diferentes**. O da Unidas conta só os carros Unidas acima de 50; o da Refran conta a ocupação **total** do pátio acima de 50 vagas. Como já havia carros Stellantis em SSA quando o contrato começou, o pátio estoura as 50 vagas da Refran muito antes de a Unidas sozinha chegar a 50. O dashboard tem um **simulador** justamente para dimensionar isso numa renegociação.

Como a medição mensal é calculada

Stellantis: somatório de `client_charge` em `vehicle_service_charges` no intervalo do mês, com dois filtros obrigatórios:

- `status <> 'cancelled'` — um terço das linhas está cancelada
- veículos com `status = 'archived'` fora da conta (são testes e registros inválidos)

A Unidas **não entra** na medição Stellantis; tem apuração própria por ocupação.

Unidas: série de `daily_occupancy` do mês filtrada por `occ_date >= contrato.starts_on`, aplicando a fórmula do piso mais excedentes.

O que nunca fazer

Três regras vieram de erros reais e custaram dinheiro ou retrabalho. Valem para qualquer feature nova.

1. Um print = um evento

Cada foto enviada pelo operador representa **um único** movimento. Ao processar uma desmobilização, lance a desmobilização e as diárias — **não** lance uma ativação retroativa só porque não existe cobrança de ativação anterior no banco. Isso é problema de outro print. Da mesma forma, ao processar uma ativação, não antecipe diárias nem saída.

Origem: 18/05/2026, veículo TZD7D87 — um único print de "Saída" gerou ativação e desmobilização ao mesmo tempo.

2. Nunca mostrar valores ao operador

As telas usadas por Bira, Diego e Marcos **não podem exibir R\$, totais, custos ou qualquer informação comercial**. Só placa, modelo, badges de tipo/movimento e data. Eles são prestadores terceirizados e não devem saber quanto a Stellantis paga nem quanto o pátio recebe.

Origem: 25/05/2026 — a tela de sucesso mostrava "Taxa Ativação R\$ 190 / Total cliente R\$ 190 / Custo parceiro R\$ 130".

3. Conferir a placa antes de lançar um extra

Fluxo obrigatório:

1. `SELECT id, plate, status FROM vehicles WHERE plate IN (...)`
2. Se não houver correspondência exata, buscar por semelhança (`ILIKE '%parte%'`) e **apresentar os candidatos**

3. Só inserir a cobrança **depois de confirmado o vehicle_id**

Uma cobrança sem veículo válido vira órfã e some da medição.

Origem: 30/05/2026 — dois extras de retirada de adesivo pedidos para placas que não existiam no banco.

Outras decisões estruturais

OS da oficina não entram na medição. Peças e serviços das ordens de serviço são faturamento separado (Proline). A planilha de OS serve apenas como fonte confiável de **datas de saída**.

A coluna "DATA DA OS" não é a data de entrada no pátio. É a data de abertura da ordem de serviço, quase sempre posterior. Usar essa coluna como entrada já produziu diárias erradas.

Conferência física é a fonte de verdade — e, entre duas evidências físicas, **listas com data prevalecem sobre fotos de posição**. Fotos podem estar desatualizadas.

Semântica das vistorias Vexsoft:

- **Desmobilização** tem vistoria na entrada (recebimento) **e** na saída → dois PDFs por carro.
- **Ativação** historicamente só tinha vistoria na **saída** (entrega do 0 km) → um PDF, que marca a **saída**, não a entrada. `km = 0` confirma que é ativação.
- Desde agosto/2026 ficou combinado com a Stellantis que a ativação passa a ter vistoria também na entrada.

Backlog vale de dezembro/2025 em diante. Todos os carros do período devem ter cobrança, preenchida com a mesma lógica e a mesma tabela de preços.

05 — Pátio PWA (operadores)

URL: `https://app.fleetflow.digital` · **Código:** `~/Desktop/Fleet Flow/patio-pwa/` · **Deploy:** Vercel, projeto `patio-pwa` **Em produção desde:** 25/05/2026

App instalável (PWA) usado por Bira, Diego e Marcos para registrar movimentações direto do celular, no pátio. Substituiu um bot de Telegram.

Stack

Framework	Next.js 14.2.15 (App Router) + TypeScript strict
UI	Tailwind 3.4 — botões e campos grandes, pensados para uso em campo
Dados	Supabase (<code>@supabase/ssr 0.5</code> , <code>supabase-js 2.45</code>)
IA	Anthropic SDK 0.32 — modelo <code>claude-sonnet-5</code>
PWA	<code>manifest.json</code> + service worker próprio (<code>public/sw.js</code> , cache v2)

O upgrade para Sonnet 5 no OCR aconteceu em 24/07/2026; antes era Haiku 4.5, que errava placas com frequência.

Estrutura

```
patio-pwa/
├── middleware.ts           gate de rotas + refresh de sessão
├── lib/supabase/
│   ├── client.ts         client de browser (anon)
│   ├── server.ts         client de server component (cookies)
│   ├── admin.ts          client service_role — hoje não usado
│   ├── middleware.ts     updateSession(): a lógica real do gate
│   └── types.ts          tipos manuais do domínio
├── app/
│   ├── login/            e-mail + senha (server action)
│   ├── home/
│   │   └── page.tsx       saudação, badge de pendências, últimas 10
├── movimentações
│   ├── actions.ts        undoMovement()
│   ├── history-list.tsx  lista + botão Desfazer
│   ├── nova/
│   │   └── pendencias/   fila de entradas faltantes
│   ├── api/ask/route.ts  assistente do dashboard (não é tela de operador)
│   ├── push-setup.tsx    opt-in de Web Push
│   └── register-sw.tsx   registra o SW em produção
├── public/
│   ├── sw.js manifest.json icon-*.png
│   ├── unidas.html refran.html  portais servidos por rewrite
├── scripts/
│   └── create-operators.mjs  cria os 3 operadores no Auth
```

```
└─ generate-icons.mjs
└─ supabase/migrations/
```

Autenticação

Supabase Auth com e-mail e senha. Três detalhes que já custaram tempo:

A server action de login não usa `redirect()`. Ela devolve `{ ok: true }` e o cliente faz `router.refresh()` seguido de `router.replace('/home')`. O `NEXT_REDIRECT` conflitava com o `await` dentro do `onSubmit`.

O middleware precisa copiar os cookies para o redirect. A função `createRedirectWithCookies()` existe porque, sem ela, o JWT refrescado pelo `getUser()` se perde e o navegador entra em loop infinito entre `/home` e `/login`. Foi um bug real, com centenas de requisições por segundo.

Autorização é em duas camadas. O middleware barra quem não está logado; a página `/home` refaz a verificação e exige `role = 'operator'` e `yard` preenchido. Ou seja, **admin não entra no PWA**.

Rotas públicas (sem sessão): `/`, `/login`, `/api/auth`, `/api/ask`, `/manifest.json`, `ícones`, `/unidas`, `/refran`.

Contas: `bira@fleetflow.app` (SSA), `diego@fleetflow.app` (REC), `marcos@fleetflow.app` (NAT). Criadas por `scripts/create-operators.mjs`; reset de senha pelo painel do Supabase.

O fluxo de uma movimentação

- | | |
|-----------------|---|
| 1. FOTO | <code>input capture=environment</code> (câmera traseira), limite de 25 MB |
| 2. COMPRESSÃO | <code>canvas</code> → maior lado 1920px, JPEG 85%
(para caber no limite de 5 MB da API de visão) |
| 3. UPLOAD | bucket privado <code>patio-evidence</code>
<code>path {yard}/{yyyy-mm-dd}/{uuid}.jpg</code> |
| 4. VISÃO | server action baixa a imagem e manda pro <code>claude-sonnet-5</code>
→ placa, modelo, marca, tipo_carro, movimento, data, confiança |
| 5. NORMALIZAÇÃO | <code>normalizePlate()</code> corrige OCR por posição |
| 6. REVISÃO | operador confere e ajusta os toggles |
| 7. CONFIRMAÇÃO | <code>saveMovement()</code> grava veículo + cobranças + histórico |
| 8. SUCESSO | placa, modelo, badges e data – sem nenhum valor em R\$ |

O OCR de placa

O maior problema prático do app. A placa Mercosul segue o padrão `LLLNLNN` — posições 1 a 3 e 5 são letras, 4, 6 e 7 são números. O modelo confunde `0↔0`, `I↔1`, `S↔5`, `B↔8`, `G↔6`, `Q↔0`, `Z↔2`.

Duas defesas:

1. **O prompt ensina o formato** e traz exemplos de erros reais já vistos (`TXP0D06` → `TXP0D06` , `TCR7B5G` → `TCR7B50`).
2. **`normalizePlate()` corrige por posição** — se a posição exige dígito e veio letra, converte; e vice-versa.

Se mesmo assim a placa não bater com o padrão Mercosul, a confiança cai para `baixa` e uma observação pede conferência ao operador. Placas no formato antigo (`LLLNNNN`) também são rebaixadas: são raras na frota e é exatamente nisso que um Mercosul mal lido se transforma.

A tela de revisão

Três toggles e quatro campos de texto:

Toggle	Quando aparece
Cliente (Stellantis / Unidas)	só para o operador de SSA
Tipo do carro (Ativação / Desmobilização)	escondido quando o cliente é Unidas
Movimento (Entrada / Saída)	sempre

Um badge colorido mostra a confiança do OCR. O estado do fluxo vive em `sessionStorage` — fechar a aba entre a revisão e a confirmação perde o lançamento (com fallback correto para o início).

O que é gravado no banco

Entrada

Veículo — busca por placa. Se existe, atualiza (`status='patio'` , pátio, data de entrada, `withdrawal_date` limpo, `notes` **sobrescrito**). Se não existe, insere, preenchendo `year` com o ano corrente e `chassi` com `PENDING-{placa}` .

Cobranças — **só Stellantis**. Os preços são lidos de `service_pricing` (não hardcoded):

Taxa		1ª diária
<code>service_key</code>	<code>ativacao</code> ou <code>desmobilizacao</code>	<code>diaria</code>
<code>client_charge</code>	190,00 ou 88,00	0,00 (carência)
<code>partner_cost</code>	123,50 ou 57,20	6,50
<code>grace_applied</code>	<code>false</code>	<code>true</code>

Unidas não gera nenhuma cobrança — a medição é por ocupação agregada.

Histórico — sempre, para os dois clientes: `ativacao_entrada` , `desmobilizacao_entrada` ou `guarda_entrada` .

Saída

Busca o veículo por placa + **cliente** + `status='patio'` . Se não achar, a mensagem é explícita: "*Confira o cliente selecionado.*"

Atualiza `withdrawal_date`, muda o status para `finalizado` e **concatena** o texto novo em `notes` com `|`. **Nenhuma cobrança é criada.**

Tela de pendências

Resolve o furo que o pipeline Vexsoft expõe: uma **saída** detectada por vistoria em um veículo que **nunca teve entrada registrada**. Sem a entrada não há data de chegada, taxa nem diárias — o faturamento fica incompleto.

A tela lista os registros de `entradas_pendentes` com status `pendente` e pede uma coisa só: **a data em que o carro entrou no pátio**. Ao salvar, o app cria o veículo já como `finalizado` (entrada e saída conhecidas), lança a taxa de ativação e **uma diária por dia** do intervalo, respeitando a carência, e baixa a pendência.

⚠ A tela **não filtra por pátio** — hoje Bira consegue ver e resolver pendências de Natal. Ver [pendências](#).

Tela de conferência de pátio (`/home/patio`) — 19/08

Posição atual do pátio do operador (placa, veículo, cliente, entrada, dias) com botões **↓ XLSX** e **↓ PDF** (bibliotecas carregadas de CDN sob demanda). Mesma lista dos botões de exportação do dashboard — serve para imprimir e conferir o pátio fisicamente. Acesso pelo atalho "📄 Conferir pátio" na home.

Auditoria física semanal (`/home/auditoria`) — 19/08

Toda segunda-feira às 8h um robô cria uma auditoria por praça (`audits` + `audit_items`, snapshot dos veículos em pátio) e o trigger do banco dispara push aos operadores. A tela mostra barra de progresso e um cartão por veículo com duas ações:

- 📷 **Tirar foto** — abre a câmera traseira, comprime localmente (máx. 1600 px, JPEG 80%) e sobe para o bucket `auditorias` (`{audit_id}/{placa}_{ts}.jpg`); o item vira `fotografado`.
- "**Não está**" — marca `nao_encontrado` (com confirmação), sinalizando divergência a investigar.

Quando todos os itens são resolvidos, a auditoria muda sozinha para `concluida`. Um banner violeta na home aparece enquanto houver auditoria pendente do pátio do operador.

Notificações push

O operador vê um botão "Ativar notificações de pendências" na home (só quando a permissão ainda está indefinida). Ao aceitar, a assinatura vai para `push_subscriptions`. Quando o pipeline insere uma linha em `entradas_pendentes`, um trigger chama a Edge Function `notify-entrada-pendente`, que dispara o push para todos os inscritos e remove assinaturas mortas.

No iOS o `PushManager` só existe com o app **instalado na tela de início** — no Safari comum o botão simplesmente não aparece.

⚠ A tabela `push_subscriptions` está **vazia**: ninguém assinou ainda, então nenhuma notificação está sendo entregue.

Botão Desfazer

Aparece nos lançamentos das **últimas 24 horas** feitos pelo **próprio operador**.

Movimento desfeito	O que acontece
Entrada	Cancela a taxa e a 1ª diária daquele dia e arquiva o veículo (<code>status='archived'</code> , o que também o tira do cron)
Saída	Devolve o veículo para <code>patio</code> e limpa <code>withdrawal_date</code> . Nenhuma cobrança é tocada

Sempre grava um registro `undo` no histórico. Nada é apagado.

O que ele não faz: não cancela diárias lançadas pelo cron em dias posteriores, não restaura as `notes` originais e não devolve o `status / client_id` anterior quando a entrada foi uma reentrada. Nesses casos, correção manual.

Variáveis de ambiente

Variável	Onde é usada
<code>NEXT_PUBLIC_SUPABASE_URL</code>	todos os clients
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	clients de browser e middleware
<code>SUPABASE_SERVICE_ROLE_KEY</code>	<code>/api/ask</code> e o script de criação de operadores
<code>ANTHROPIC_API_KEY</code>	visão (OCR) e assistente
<code>DASH_PASSWORD_SHA256</code>	(<i>opcional</i>) autenticação do assistente
<code>ASK_MODEL</code>	(<i>opcional</i>) troca o modelo do assistente

Todas configuradas em **Production** e **Preview** na Vercel.

Deploy

```
cd ~/Desktop/"FLeet Flow"/patio-pwa
npx vercel deploy --prod
```

Não há repositório Git nem CI: o deploy é feito da pasta local. O Vercel CLI builda remotamente, o que também contorna o problema de arquivos "placeholder" do iCloud.

Instalação no celular

- **iOS:** Safari → Compartilhar → "Adicionar à Tela de Início"
- **Android:** Chrome → menu → "Instalar app"

O service worker é *network-first* (o operador precisa de dado fresco) e mantém em cache apenas o shell para uma tela mínima offline.

06 — Dashboard interno e assistente de IA

URL: `https://dash.fleetflow.digital` · **Repo:** `chicocbrandao/fleetflow-dashboard` (este) · **Local:** `~/projetos/fleetflow-dashboard/`

Um único arquivo `index.html` servido pelo GitHub Pages. Sem build, sem framework, sem backend próprio: todas as bibliotecas vêm de CDN e os dados vêm direto do Supabase com a chave anônima.

Bibliotecas

Lib	Uso
<code>@supabase/supabase-js</code>	leitura das tabelas
<code>chart.js</code>	gráficos
<code>xlsx</code> (SheetJS)	export Excel
<code>jspdf</code> + <code>jspdf-autotable</code>	export PDF

Nota histórica: o jsPDF é carregado do **jsDelivr**. A URL antiga do cdnjs (2.5.2) passou a devolver 404 em 11/08/2026 e quebrou o export PDF sem aviso. Se o PDF parar de funcionar, o primeiro suspeito é o CDN.

Acesso

Uma senha comparada por SHA-256 no próprio JavaScript. Não é autenticação forte — é um portão para não deixar a operação exposta em uma URL pública. A sessão fica em `sessionStorage`.

Desde 14/08/2026 o login também guarda a senha em `sessionStorage`, porque é ela que autentica as chamadas ao assistente.

Seções

KPIs — total de veículos, em pátio (com quebra por praça), finalizados, ativações e desmobilizações do mês corrente, e a medição do mês.

Gráficos — distribuição por praça e por status.

Unidas x Refran (Salvador) — carros Unidas hoje, ocupação total do pátio, fatura Unidas do mês, repasse Refran do mês e a margem da guarda. Abaixo, um gráfico empilhado de 30 dias (Stellantis + Unidas) com a linha das 50 vagas contratadas, e um **simulador** que projeta custo, receita e margem para diferentes cenários de vagas contratadas, mensalidade e valor de excedente — a ferramenta para renegociar com a Refran.

Consulta de veículo — busca por placa e mostra o ciclo completo: praça, tipo, entrada, saída, dias totais, dias cobrados, custódia, serviço e total estimado.

Veículos em pátio — tabela filtrável por praça, com dias de permanência e custo acumulado.

Medição mensal — seletor de mês, filtro por tipo (todos / ativação / desmobilização), totalizadores e a tabela detalhada por veículo, com **export para Excel e PDF**. A medição **exclui a Unidas** (que tem apuração própria).

Cálculos feitos no navegador

O dashboard não lê `vehicle_service_charges` para montar a medição — ele **recalcula** a partir de `vehicles`, aplicando as regras direto no JavaScript. Isso é rápido e simples, mas significa que **o dashboard e o banco podem divergir** se as cobranças tiverem sido ajustadas manualmente. Para o número contratual, a fonte é sempre `vehicle_service_charges`; para o número gerencial do dia a dia, o dashboard basta.

Constantes no topo do arquivo: carência de 7 dias, diária R\$ 10, ativação R\$ 190, desmobilização R\$ 88, diária Unidas R\$ 6.

KPIs de "hoje" contam veículos ao vivo, não o snapshot: o `daily_occupancy` só é atualizado às 07:05 e não enxerga lançamentos feitos durante o dia nem retroativos. Essa distinção foi introduzida em 29/07/2026, depois de o painel mostrar 13 carros Unidas quando havia 20.

Correções aplicadas (histórico)

Data	Correção
24/07	<code>parseType</code> passou a reconhecer <code>GUARDA</code> e a respeitar a ordem das palavras em <code>notes</code>
—	<code>daysBetween</code> passou a contar o dia de entrada (+1)
—	<code>custo</code> passou a somar a taxa, não só as diárias
29/07	KPIs de "hoje" passaram a contar veículos ao vivo
—	medição mensal passou a excluir a Unidas
11/08	jsPDF migrado para o jsDelivr
14/08	assistente de IA

Como atualizar

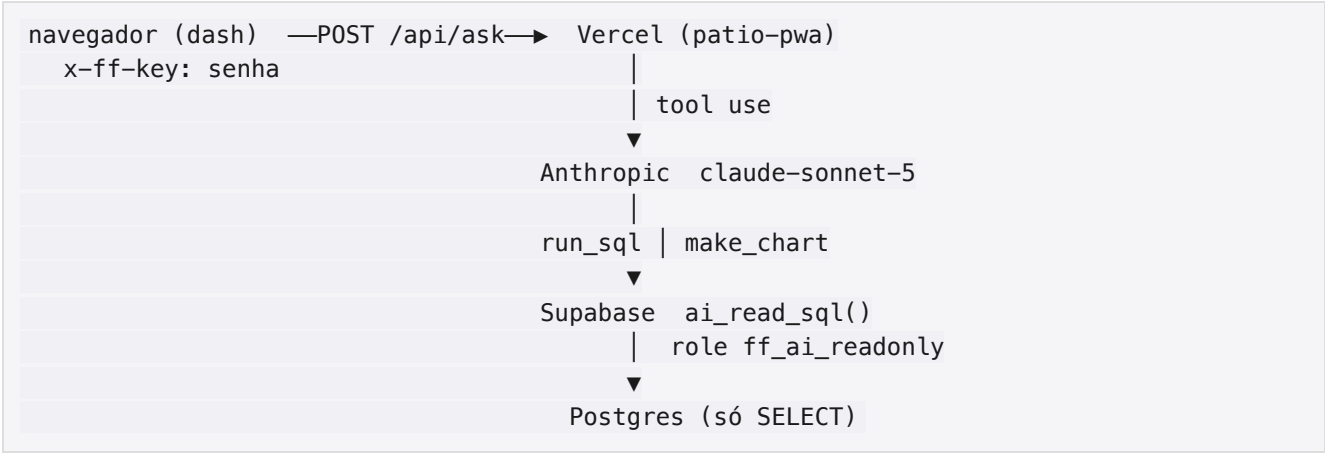
```
cd ~/projetos/fleetflow-dashboard
# edite index.html
git add -A && git commit -m "... " && git push origin main
# o GitHub Pages publica em ~2 minutos
```

Assistente de IA

Adicionado em 14/08/2026. Um box de chat no dashboard que responde perguntas sobre a operação e monta relatórios consultando o banco.

Por que não fala direto com a Anthropic

O `index.html` é público — uma chave de API no HTML seria uma chave vazada. O chat conversa com a rota `/api/ask` dentro do PWA, que roda na Vercel e é onde `ANTHROPIC_API_KEY` e `SUPABASE_SERVICE_ROLE_KEY` já existiam. Nenhuma variável nova precisou ser criada.



Segurança em três camadas

1. **Acesso** — o header `x-ff-key` leva a senha do dashboard; o servidor compara o SHA-256 com o hash conhecido. Quem tem a senha do dash tem o assistente, e nada mais.
2. **CORS** — só `dash.fleetflow.digital`, `chicocbrandao.github.io` e `localhost`.
3. **Banco** — toda consulta passa por `ai_read_sql()`, que troca para o papel `ff_ai_readonly` (sem nenhum privilégio de escrita, sem acesso a `auth` ou `storage`), limita a 2.000 linhas e derruba a consulta em 20 segundos. CTEs que modificam dados são recusadas pelo próprio Postgres.

Não é possível, por construção, o assistente alterar qualquer coisa no banco.

Ferramentas do modelo

Ferramenta	O que faz
<code>run_sql(sql, is_report, title)</code>	Executa um SELECT. Quando <code>is_report</code> é verdadeiro, aquele resultado vira a tabela baixável
<code>make_chart(type, title, labels, series)</code>	Renderiza um gráfico Chart.js no chat

O modelo pode encadear várias consultas — explorar, errar, ler a mensagem de erro e corrigir — antes de responder. O limite é de 12 rodadas por pergunta.

O que o modelo sabe

O *system prompt* carrega, além do schema das 10 tabelas úteis:

- os UUIDs de Stellantis e Unidas
- a matriz de cobrança e a carência de 7 dias
- os contratos Unidas e Refran
- o formato do campo `notes` e a regra de "vale a primeira palavra"
- as armadilhas: excluir `archived`, excluir `status='cancelled'`, Unidas fora da medição Stellantis, `yard` nulo em registro antigo, `withdrawal_date` como `timestampz`

É esse contexto que faz a diferença entre um gerador de SQL genérico e um assistente que acerta o número da medição.

Saídas

A resposta do chat pode trazer três coisas: o texto, uma **tabela** (com botões de Excel e PDF que exportam o **conjunto completo**, não só as 50 linhas de prévia) e um **gráfico**.

Custo

Aproximadamente **R\$ 0,05 a R\$ 0,30 por pergunta** com `claude-sonnet-5`, dependendo de quantas consultas o modelo precisa rodar. A variável `ASK_MODEL` permite trocar o modelo sem mexer no código.

Exemplos de pergunta

- *"Medição de agosto por praça"*
- *"Carros parados há mais de 30 dias"*
- *"Faturamento mês a mês em 2026, com gráfico"*
- *"Margem da guarda Unidas × Refran neste mês"*
- *"Vistorias Vexsoft sem lançamento no sistema"*

07 — Integrações

Pipeline Vexsoft — ingestão automática de vistorias

Em operação desde 13/08/2026. É hoje a **entrada principal de dados da Stellantis**; o PWA virou backup para esse cliente (com deduplicação automática). A Unidas continua 100% no PWA.

A decisão de fundo

Toda movimentação de veículo Stellantis passa por uma vistoria da **Vexsoft**, que gera um PDF e envia um e-mail. Em vez de depender de o operador lembrar de lançar no app, o sistema passou a **ler a vistoria como fonte primária**. Junto com isso ficou combinado com a Stellantis que as ativações passariam a ter vistoria também na **entrada** — antes só existia a da entrega.

Escopo: REC, SSA e NAT.

Como funciona



```

| ATIVAÇÃO SEM entrada no PWA (19/08) ..... NÃO lança: insere em
|   entradas_pendentes → push ao operador → acao aguardando_entrada_pwa
|   (por determinação da Stellantis, entrada de ativação é só pelo PWA)
| movimento já registrado no PWA (±2 dias) . duplicado_pwa → não lança
| conflito / placa inválida / endereço . excecacao → push, não lança
|
| ▼
registra a decisão em vexsoft_ingest

```

Onde o robô roda (desde 19/08/2026)

Script Python **no Mac mini** (`macmini-server` , acesso por `ssh mini` — ver memory `reference_acesso_mac_mini`): `/Users/agent/fleetflow/robos/robo_vexsoft.py` , agendado pelo `launchd` (`br.fleetflow.robo-vexsoft`) **de hora em hora no minuto 35**. Cópia de trabalho dos scripts também em `Desktop/FLeet Flow/robos/` (iCloud) no MacBook. Ele lê a caixa `chico@clampatrimonial.com.br` (AZPark Mail, onde chegam os e-mails do Vexsoft) via IMAP com senha de app, extrai os campos do PDF com `pypdf` e grava no Supabase via REST. A praça é determinada pelo **CEP/cidade do endereço da vistoria** (4xxxx→SSA, 50–56xxx→REC, 59xxx→NAT). Logs em `robos/logs/robo_vexsoft.log` . Exceções disparam push (`push-broadcast`).

Transição: a tarefa agendada na nuvem (cron `20 * * * * UTC`) roda **em paralelo até ~26/08/2026** como rede de segurança e depois será desativada. O dedupe por `gmail_message_id` e por `vistoria_id` garante que os dois nunca lançam a mesma vistoria duas vezes.

Rastreamento

Cada e-mail processado vira uma linha em `vexsoft_ingest` , com a decisão no campo `acao` :

<code>acao</code>	Significa
<code>entrada_criada</code>	veículo entrou (ou reentrou) e a cobrança foi lançada
<code>saida_lancada</code>	ciclo fechado
<code>duplicado_pwa</code>	o operador já tinha lançado; nada foi feito
<code>aguardando_entrada_pwa</code>	ativação sem entrada — pendência criada e push enviado ao operador
<code>excecacao</code>	precisa de decisão humana

`gmail_message_id` é UNIQUE e o robô também confere o `vistoria_id` — é o que garante que reprocessar a caixa (ou rodar dois robôs em paralelo) não duplica nada.

Limitações conhecidas

- **E-mails com mais de 3 dias não são processados.** Retroativos continuam sendo trabalho manual.
- Se a Vexsoft mudar o formato do e-mail ou do PDF, o robô passa a gerar exceções — o conserto é ajustar as expressões regulares em `robos/robo_vexsoft.py` (funções `parse_email` e `read_pdf`).
- O pipeline **não baixa nem arquiva os PDFs assinados**. Se uma auditoria exigir os comprovantes, isso ainda é feito à parte.

Entradas pendentes

Quando a vistoria indica uma **saída** de um veículo sem entrada registrada, o pipeline não inventa a data: cria uma linha em `entradas_pendentes` e notifica. O operador informa a data pela tela de pendências do PWA, que então cria o veículo, a taxa e todas as diárias do intervalo. Ver [05](#).

Portais externos — Unidas e Refran

Duas páginas de prestação de contas, uma por contraparte, publicadas desde 24/07/2026.

	Unidas	Refran
URL	<code>unidas.fleetflow.digital</code>	<code>refran.fleetflow.digital</code>
Também em	<code>app.fleetflow.digital/unidas</code>	<code>app.fleetflow.digital/refran</code>
Papel	cliente (receita)	fornecedor (custo)
Edge Function	<code>dashboard-unidas</code>	<code>dashboard-refran</code>
Projeto Vercel	<code>fleetflow-unidas</code>	<code>fleetflow-refran</code>

Stack: HTML estático único, sem framework e sem dependência externa — o gráfico de barras é **SVG desenhado à mão**. Tema claro/escuro automático.

Acesso: token na query string (`?t=...`). Sem token, a página mostra um portão pedindo o código. A validação real acontece na Edge Function.

O que a Unidas vê: carros no pátio agora, média diária do mês, dias acima do mínimo de 50, fatura parcial, gráfico de ocupação com a parte excedente destacada, lista de carros no pátio (placa, modelo, entrada, dias) e a memória de cálculo dia a dia. Navegação mês a mês.

O que a Refran vê: ocupação de hoje, vagas contratadas, carro-dias excedentes no mês e o repasse parcial — com a ocupação **somando todos os clientes. Não vê placas, nomes de clientes nem receita**, por decisão de projeto.

⚠ Duplicação: os arquivos em `fleetflow-unidas/index.html` e `patio-pwa/public/unidas.html` são idênticos byte a byte (idem para a Refran). São duas cópias para manter em sincronia e dois caminhos de deploy. Vale escolher um canal único.

Notificações push

Peça	Onde
Opt-in do operador	<code>app/push-setup.tsx</code> no PWA
Registro das assinaturas	tabela <code>push_subscriptions</code>
Disparo	trigger <code>trg_notify_entrada_pendente</code> em <code>entradas_pendentes</code> → função <code>notify_entrada_pendente()</code> → Edge Function <code>notify-entrada-pendente</code>
Entrega	Web Push (VAPID)

A notificação usa tag: 'entrada-pendente', então uma nova substitui a anterior em vez de empilhar. Assinaturas que respondem 404 ou 410 são apagadas automaticamente.

Estado atual: a tabela está vazia. Nenhum operador assinou, então nenhuma notificação chega. Ver [10](#).

Bot de Telegram — legado

Status: superado, sem registro de desligamento formal.

Bot standalone em Node.js (`node-telegram-bot-api` + Gemini 2.5 Flash + API REST do Supabase), rodando no Mac Mini em `~/projetos/fleetflow-bot/`. Recebia screenshots de WhatsApp encaminhados, extraía placa/modelo/tipo/data com o Gemini e inseria no Supabase. Tinha comandos `/status`, `/buscar PLACA` e `/help`, além de retry com backoff para as respostas 429 e 503 do free tier.

Foi criado em meados de 2026 para substituir o **Neo/OpenClaw**, que parou de funcionar em maio (loop de timeout de rede, configuração desatualizada, rate limit do relay).

O que o substituiu: o **PWA**, desde 25/05/2026, para a operação diária; e o **pipeline Vexsoft**, desde 13/08/2026, para a Stellantis.

Pendências relacionadas: nunca foi criado o LaunchAgent para auto-start, e o token do bot e a chave do Gemini estão registrados em texto claro na documentação interna — devem ser rotacionados quando o bot for oficialmente aposentado.

Script `baixar_vex.sh` — legado

Script Bash gerado automaticamente que baixava os 136 comprovantes originais de vistoria de abril a julho/2026 e os concatenava em quatro PDFs, um por grupo de medição. Artefato pontual de fechamento, não um componente do sistema: as URLs são fixas e específicas daquelas vistorias.

Serve como referência para o dia em que for preciso arquivar comprovantes assinados de forma automática — algo que o pipeline atual não faz.

08 — Operação e runbook

Deploy

Dashboard (este repositório)

```
cd ~/projetos/fleetflow-dashboard
# edite index.html ou docs/
git add -A && git commit -m "... " && git push origin main
```

O GitHub Pages publica em cerca de 2 minutos. O domínio customizado está fixado via API do GitHub (o arquivo `CNAME` sozinho não bastou).

Pátio PWA

```
cd ~/Desktop/"FLEet Flow"/patio-pwa
npx vercel deploy --prod
```

Não há Git nem CI. O build acontece na Vercel, o que também contorna arquivos "placeholder" do iCloud. Variáveis de ambiente pelo painel da Vercel (Production e Preview).

Portais Unidas / Refran

```
cd ~/Desktop/"FLEet Flow"/fleetflow-unidas # ou fleetflow-refran
npx vercel deploy --prod
```

Lembre-se: existe uma cópia idêntica em `patio-pwa/public/`. Alterou uma, atualize a outra.

Banco

Migrations pelo MCP do Supabase ou pelo painel. As migrations do PWA ficam versionadas em `patio-pwa/supabase/migrations/`.

Monitoramento

O cron rodou?

```
select jobid, status, start_time, return_message
from cron.job_run_details
where jobid = (select jobid from cron.job where jobname = 'launch-daily-charges')
order by start_time desc
limit 10;
```

Esperado: `succeeded`, uma execução por dia às 10:05 UTC, duração abaixo de 1 s.

As diárias de hoje foram lançadas?

```
select count(*), sum(client_charge)
from vehicle_service_charges
where service_key = 'diaria'
  and charge_date = (now() at time zone 'America/Bahia')::date
  and status <> 'cancelled';
```

O número deve bater com a quantidade de veículos Stellantis em pátio.

O pipeline Vexsoft está processando?

```
select acao, count(*), max(created_at)
from vexsoft_ingest
group by acao
order by 2 desc;
```

Preste atenção nas linhas com `acao = 'excecao'` — cada uma precisa de decisão humana.

Tem pendência aberta?

```
select plate, yard, saida_date, created_at
from entradas_pendentes
where status = 'pendente'
order by created_at;
```

Ocupação de hoje (ao vivo, não o snapshot)

```
select coalesce(yard, '?') as patio,
       count(*) filter (where client_id = '167748aa-2fee-49df-9b0f-140919bfbf7c') as
stellantis,
       count(*) filter (where client_id = 'fea28c45-8ca5-4550-98ad-3f9e78b7a120') as
unidas,
       count(*) as total
from vehicles
where status = 'patio'
group by 1 order by 4 desc;
```

Rotinas automáticas (visão consolidada)

Quando	O quê	Onde conferir
Diário 07:05 (Bahia)	<code>launch_daily_charges()</code> — diárias Stellantis + snapshot de ocupação (janela de 7 dias)	<code>vehicle_service_charges</code> / <code>daily_occupancy</code>
A cada hora (min 35)	Robô Vexsoft (Mac mini , <code>launchd</code> <code>br.fleetflow. robo-vexsoft</code>) processa e-mails de vistoria	<code>vexsoft_ingest</code> / <code>robos/logs/robo_vexsoft.log</code>

Quando	O quê	Onde conferir
Segunda 08:00 (Bahia)	Robô de auditoria (Mac mini , launchd <code>br.fleetflow.robô-auditoria</code>) cria <code>audits / audit_items</code> por praça e o trigger dispara push aos PWAs (<code>push-broadcast</code>)	<code>audits / audit_items</code> / fotos no bucket <code>auditorias</code>

Os dois robôs rodam no **Mac mini** (`macmini-server`) desde 19/08/2026, no usuário `agent` , em `/Users/agent/fleetflow/robos/` (acesso: `ssh mini` ; cópia dos scripts também em `Desktop/FLeet Flow/robos/` no MacBook). O mini já é servidor 24/7 (`pmset sleep 0`). As tarefas agendadas equivalentes na nuvem ficam ativas em paralelo até ~26/08/2026 e depois serão desativadas.

Alerta se um robô parar (watchdog)

Cada rodada bem-sucedida atualiza um **sinal de vida** na tabela `heartbeats` . Dois vigias independentes do Mac mini conferem esse sinal:

Vigia	Onde roda	Frequência	Dispara quando	Alerta por
<code>check_robot_heartbeats()</code>	<code>pg_cron</code> no Supabase	a cada 30 min	Vexsoft >2h30 sem rodar; auditoria >7d6h	web push aos PWAs (<code>push-broadcast</code>), no máx. 1 a cada 6h
Vigia diário	Tarefa agendada na nuvem, 09:00 (Bahia)	1×/dia	robô parado, >3 exceções em 24h ou pendência de entrada >48h	notificação push do app Claude no celular do gestor

Para receber o web push do primeiro vigia é preciso ter tocado em "🔔 Ativar notificações" no PWA (tabela `push_subscriptions`). O segundo vigia não depende disso.

Auditoria física semanal

Criada em 19/08. O operador recebe o push na segunda, abre `/home/auditoria` e fotografa carro a carro (ou marca "Não está"). Acompanhamento do gestor:

```
-- progresso da semana
select a.yard, a.status, a.total_items,
       count(*) filter (where i.status='fotografado') as fotografados,
       count(*) filter (where i.status='nao_encontrado') as nao_encontrados
from audits a join audit_items i on i.audit_id = a.id
where a.week_start = date_trunc('week', current_date)::date
group by 1,2,3;
-- divergências (carro no sistema que não está no pátio)
select a.yard, i.plate, i.model from audit_items i
join audits a on a.id = i.audit_id
where i.status = 'nao_encontrado' order by a.week_start desc;
```

Um `nao_encontrado` é o gatilho para investigar saída não registrada (mesmo playbook das conciliações do capítulo 09).

Conferência de pátio (impressão)

Dashboard → seção "Veículos em Pátio" → botões **XLSX/PDF** (a coluna "Conferido ✓" sai em branco para marcar no papel). O mesmo existe no PWA em `/home/patio`.

Tarefas comuns

Fechar a medição do mês (Stellantis)

```
select coalesce(v.yard, '?') as patio,
       count(distinct v.id)           as veiculos,
       sum(c.client_charge)           as receita,
       sum(c.partner_cost)            as custo,
       sum(c.client_charge - c.partner_cost) as margem
from vehicle_service_charges c
join vehicles v on v.id = c.vehicle_id
where c.charge_date >= date '2026-08-01'
     and c.charge_date < date '2026-09-01'
     and c.status <> 'cancelled'
     and v.status <> 'archived'
     and v.client_id = '167748aa-2fee-49df-9b0f-140919bfbf7c'
group by 1 order by 3 desc;
```

Ou, mais simples: peça ao assistente do dashboard e baixe em Excel.

Fatura da Unidas no mês

```
with c as (select * from supply_contracts where code = 'unidas_ssa')
select c.base_monthly
      + coalesce(sum(greatest(0, o.car_count - c.included_units)), 0) * c.excess_rate as
fatura,
      coalesce(sum(greatest(0, o.car_count - c.included_units)), 0) as
carro_dias_excedentes
from c
left join daily_occupancy o
  on o.yard = 'SSA' and o.client_id = c.client_id
  and o.occ_date >= greatest(date '2026-08-01', c.starts_on)
  and o.occ_date < date '2026-09-01'
group by c.base_monthly, c.excess_rate;
```

Lançar um extra

Sempre confira a placa primeiro — ver [regra 3](#).

```
-- 1. achar o veículo
select id, plate, status, yard from vehicles where plate = 'ABC1D23';

-- 2. só depois de confirmado o id:
insert into vehicle_service_charges
  (vehicle_id, pricing_id, service_key, client_charge, partner_cost,
   charge_date, days_count, grace_applied, status, client_id, notes)
select :vehicle_id, p.id, p.service_key, p.client_price, p.partner_price,
       current_date, 1, false, 'pending', v.client_id, 'Extra lançado pelo admin'
from service_pricing p, vehicles v
```

```
where p.service_key = 'remocao_plotagem' and p.is_active  
and v.id = :vehicle_id;
```

Corrigir uma data de saída

```
update vehicles  
  set withdrawal_date = date '2026-07-22',  
      status = 'finalizado'  
where plate = 'ABC1D23';  
  
-- e cancelar as diárias posteriores à saída real  
update vehicle_service_charges  
  set status = 'cancelled',  
      notes = coalesce(notes, '') || ' | CANCELADA: pós-saída real'  
where vehicle_id = (select id from vehicles where plate = 'ABC1D23')  
  and service_key = 'diaria'  
  and charge_date > date '2026-07-22'  
  and status <> 'cancelled';
```

Criar um operador novo

Edite `scripts/create-operators.mjs` com o e-mail e o pátio e rode `node scripts/create-operators.mjs`. O script é idempotente e imprime a senha temporária.

Pausar ou reagendar o cron

```
select cron.alter_job(2, schedule := '5 10 * * *'); -- reagendar  
update cron.job set active = false where jobid = 2; -- pausar
```

Troubleshooting

Sintoma	Causa provável	O que fazer
Export PDF do dashboard não faz nada	CDN do jsPDF fora do ar ou versão removida	Trocar a URL do script (já aconteceu com o cdnjs em 11/08)
Assistente responde "Acesso negado"	Senha errada, ou sessão sem <code>ff_pw</code>	Recarregar a página e entrar de novo
Assistente dá erro de CORS	Origem não está na lista da rota <code>/api/ask</code>	Adicionar o domínio em <code>ALLOWED_ORIGINS</code>
Assistente devolve 307 / cai no login	<code>/api/ask</code> fora da lista de rotas públicas do middleware	Conferir <code>lib/supabase/middleware.ts</code>
Loop infinito de login no PWA	Cookies perdidos no redirect do middleware	<code>createRedirectWithCookies()</code> precisa copiar os cookies

Sintoma	Causa provável	O que fazer
Diárias duplicadas em um dia	Alguém inseriu diária fora do cron	Não existe UNIQUE no banco: cancelar as duplicadas à mão
Dashboard mostra menos carros que o real	Lançamento retroativo fora da janela de 7 dias do snapshot	Recalcular <code>daily_occupancy</code> do período
Saída aparece um dia adiantada	Conversão de fuso em <code>withdrawal_date</code>	Usar <code>withdrawal_date::date</code> sem <code>at time zone</code>
OCR erra a placa	Foto ruim ou caractere ambíguo	O operador corrige na tela de revisão; a confiança "baixa" é o alerta
Veículo não encontrado na saída	Cliente errado no seletor (Stellantis × Unidas)	Conferir o toggle; a busca filtra por cliente
Push não chega	<code>push_subscriptions</code> está vazia	Operador precisa aceitar a notificação no PWA instalado
Robô Vexsoft não roda no horário	Mac mini fora do ar, ou agente descarregado	<pre>ssh mini 'tail -20 fleetflow/robos/logs/robo_vexsoft.log; launchctl list \ grep fleetflow'; recarregar com launchctl unload && load do plist em ~/Library/LaunchAgents/</pre>
Robô Vexsoft: erro de login no Gmail	Senha de app revogada/trocada	Criar nova senha de app na conta <code>chico@clampatrimonial.com.br</code> e atualizar <code>GMAIL_APP_PASSWORD</code> em <code>/Users/agent/fleetflow/robos/.env</code> no mini
Robô lança tudo como exceção	Vexsoft mudou o formato do e-mail/PDF	Ajustar regexes em <code>robos/robo_vexsoft.py</code> (<code>parse_email / read_pdf</code>)

Gotchas do ambiente local

- A pasta `Fleet Flow` está no **iCloud**. Arquivos podem estar como *placeholder* e quebrar builds locais — materialize com `cat` antes, ou deixe a Vercel buildar remotamente.
- O `patio-pwa` **não é um repositório Git**. Não existe histórico nem backup fora da Vercel.
- Nunca recrie policies de RLS recursivas em `profiles`. A policy `Admins can view all profiles` custou quatro horas de depuração e foi removida.

09 — Histórico e conciliações

Linha do tempo

Data	Marco
até ~mai/2026	Legado. Dados vindos do bot Neo/OpenClaw e do projeto ProLine. Base com registros parados: 31 veículos legados, 24 deles como <code>patio</code> havia mais de 60 dias. Dezembro/2025 a março/2026 ficaram sem nenhuma cobrança lançada — só descoberto em agosto
mai/2026	O Neo/OpenClaw para de funcionar (loop de timeout, configuração desatualizada, rate limit do relay)
18/05/2026	Incidente TZD7D87 : um único print de "Saída" gera ativação e desmobilização ao mesmo tempo → nasce a regra 1 print = 1 evento
25/05/2026	PWA em produção. Migrations de RLS e papel <code>operator</code> , coluna <code>yard</code> , cron de diárias, três operadores criados, bucket de evidências, deploy na Vercel + DNS. A nova matriz <code>tipo_carro</code> × <code>movimento</code> substitui as regras antigas. Valores em R\$ removidos da tela do operador
30/05/2026	Dois extras de adesivo pedidos para placas inexistentes no banco → nasce a regra de conferir a placa antes de lançar extra
entre mai e jul	Bot de Telegram standalone construído para substituir o Neo
24/07/2026	Contratos Unidas + Refran entregues — as sete fases: banco, PWA, cron, portal Unidas, portal Refran, dashboard interno, teste ponta a ponta. Subdomínios verificados. No mesmo dia, conferências físicas em SSA e REC
27/07/2026	Início da vigência dos contratos Unidas e Refran
25–28/07/2026	Bira registra 20 carros Unidas em SSA (sete lançados retroativamente em 28/07)
29/07/2026	Correção do dashboard , que mostrava 13 em vez de 20: snapshots recalculados, cron passa a recalcular janela deslizante de 7 dias, KPIs de "hoje" passam a contar veículos ao vivo
11/08/2026	Regularização de REC com base nas vistorias Vexsoft e aplicação das listas de saídas físicas
13/08/2026	Pipeline Vexsoft em operação. O PWA vira backup para a Stellantis
14/08/2026	Conciliação Vexsoft × FleetFlow (abril a agosto) e fechamento da medição abr–jul em R\$ 27.158
14/08/2026	Assistente de IA no dashboard

Os episódios de conciliação

Quatro rodadas de acerto entre o que o banco dizia e o que o pátio tinha de fato. Valem como documentação porque explicam **por que um terço das cobranças está cancelada** e por que certas regras existem.

Conferência física de Salvador — 24/07/2026

A partir de fotos de posição das 11h29 (16 carros):

- **Cinco placas corrigidas por erro de OCR:** TDY8176→TDY8I76 , TCR7B5G→TCR7B50 , TXP0D06→TXP0D06 , SYQ5E5Q→SYQ5E50 , 0MZ7C49→QMZ7C47
- **12 saídas lançadas** com data provisória e **duas entradas criadas** (Rampage e Renegade, desmobilização)

Detalhe instrutivo: QMZ7C49 existia mesmo, como **outro** veículo, já fechado em 19/06 pelo PWA. Os dois carros eram reais — a correção de OCR não os fundiu.

Conferência física de Recife — julho/2026

Diego contou **13 veículos** no pátio; o sistema tinha **17** como **patio**.

- 7 conferiam
- 2 eram erro de placa (TYH8D65 × TVH8D65 , V↔Y; TDS1B66 × TDZ1B66 , S↔Z)
- 4 estavam no pátio e não no sistema
- 8 estavam no sistema e não no pátio — provavelmente saíram sem registro

Ficou estabelecido que **a conferência física é a fonte de verdade**. O impacto financeiro veio nas rodadas seguintes.

Regularização de Recife — 11/08/2026

(a) Retroativos a partir das vistorias. Dos 15 carros da planilha do Diego, sete não estavam no sistema. Foram lançados com entrada e saída reais, gerando **R\$ 718 de taxas e R\$ 720 de custódia**.

O caso dos dois Citroën Basalt merece nota: a vistoria de 18/05 era **saída**, não entrada. O TYG8G53 estava como **patio** desde 11/05 acumulando diárias — **81 diárias erradas foram removidas**. Foi aqui que se confirmou a regra: *para ativação, a vistoria marca a saída*.

Outra lição: a "entrada 20/05" que aparecia na planilha era **data de abertura de OS**, não data física.

Impacto: cerca de R\$ 1.818 recuperados na medição de abril a junho.

(b) Listas de saídas físicas. Dezoito datas corrigidas em SSA e REC, com nota **CANCELADA: pós-saída real** (conferência 11/08).

Impacto: 1.122 diárias canceladas = -R\$ 10.860.

Um conflito interessante: SYM5E95 aparecia nas fotos de 24/07, mas a lista dizia saída em 22/07.

Prevaleceu a lista — daí a regra de que listas com data valem mais que fotos de posição.

Posição pós-correção: SSA com 34 carros (14 Stellantis + 20 Unidas), REC com 13.

Conciliação Vexsoft x FleetFlow — 14/08/2026

Comparação de **144 vistorias** de abril a 14/08 contra os movimentos do sistema.

Diagnóstico:

- **30 vistorias sem nenhum movimento correspondente** (janela de ±3 dias)
- 27 movimentos sem vistoria — a maioria são entradas de ativação, que historicamente não tinham vistoria (esperado)

- **Recife é o furo grande:** 13 das 14 vistorias órfãs de julho são de REC, com 10 carros presos como **patio**, o mais antigo desde 09/03
- **Natal:** uma vistoria órfã e **zero movimento no sistema desde março**
- **Salvador está alinhada** — Bira lança pelo PWA de forma consistente
- **Dezembro/2025 a março/2026: zero cobranças** para cerca de 30 veículos cadastrados

A planilha de fechamento da Proline (recebida no mesmo dia) trouxe uma distinção que virou regra: a coluna **"DATA DA SAÍDA" é confiável** e bate com as vistorias; a coluna **"DATA DA OS" não é a data de entrada** no pátio.

Correções aplicadas:

Item	Efeito
Placas corrigidas: OMY5C40→QMY5C40 , TCH2B52→TCW2B52 , TZ04A91→TZ04A91	—
SINGH34 arquivado (duplicata de OCR de SIN0H34)	–R\$ 190 de taxa indevida
QMY5C40 fechado em 15/07	–R\$ 300 (30 diárias)
TDZ4D72 fechado em 17/07	–R\$ 280 (28 diárias); taxa de R\$ 190 mantida
TEJ3B07 com entrada corrigida de 31/07 para 22/06	40 diárias refeitas
Três retroativos criados em REC	taxas + diárias completas

Medição consolidada após as correções:

Mês	Valor
Abril	R\$ 4.152
Maio	R\$ 5.500
Junho	R\$ 8.968
Julho	R\$ 8.450
Abril a julho	R\$ 27.070

O total anterior era R\$ 26.264 — as conciliações **augmentaram** a medição, apesar de terem cancelado mais de mil diárias. O que se recuperou em lançamentos faltantes superou o que se devolveu em cobrança indevida.

Limpeza de duplicatas — 14/08/2026

Ao criar o índice único de cobranças, apareceram três linhas duplicadas que nenhuma conciliação anterior tinha pegado, porque não eram erro de data e sim de gravação:

- **TEY8C68** (REC, 09/06) — a mesma desmobilização foi lançada duas vezes no PWA, com seis minutos de diferença e duas fotos distintas. Cobrou R\$ 88 a mais do cliente e R\$ 63,70 a mais do parceiro.
- **QMZ7C56** (SSA, 22/05) — a diária de entrada foi lançada duas vezes, com três semanas de intervalo. R\$ 6,50 a mais no parceiro.

As três foram canceladas e junho caiu de R\$ 9.056 para **R\$ 8.968**. O índice único impede que se repita.

Contradições registradas

Pontos onde os registros históricos divergem. Ficam documentados para evitar retrabalho:

TVH8D65 × **TYH8D65** . A conferência de julho tratou como erro de digitação; a conciliação de 14/08 concluiu que são a **primeira e a segunda passagem do mesmo carro** — registros legítimos distintos, que a constraint UNIQUE de `plate` impede de coexistir. **Prevalece a versão de 14/08.**

TDS8G29 . Em julho aparece como "no pátio físico e não no sistema"; em 14/08 aparece como entrada por vistoria em 11/05. Ou foi cadastrado no intervalo sem registro, ou uma das leituras está errada.

Horário do cron. Registros de maio citam 03:05 UTC; os de julho, 10:05 UTC. **O valor atual no banco é 5 10 * * *** — o horário foi mudado para casar com a regra "todo dia às 7h" dos contratos.

Status do bot de Telegram. Descrito como ativo em julho, marcado para desligamento em maio. Não há registro de decisão final.

10 — Pendências e riscos

Situação em **14/08/2026**. Ordenado por gravidade dentro de cada bloco.

Riscos de infraestrutura

● O código do PWA existe em um lugar só — resolvido em 14/08

`~/Desktop/Fleet Flow/patio-pwa/` não era repositório Git: não havia histórico nem backup fora do build da Vercel.

Versionado em 14/08/2026 (commit inicial `b6c2aa0`, 58 arquivos). O `.gitignore` que já existia cobre `node_modules`, `.next`, `.vercel` e `.env*.local` — confirmado que nenhum segredo entrou; o único JWT versionado é a chave anônima, que já é pública. **Falta criar o repositório remoto — privado — e dar o primeiro push.**

● Segredos em texto claro na pasta do Desktop

`FleetFlow_Credenciais_AI_Agents.docx`, `patio-pwa/.env.local` e `proline/.env.local` estão numa pasta sincronizada por iCloud. Migrar para um gerenciador de senhas.

● Tokens e chaves VAPID como constantes no código

Os tokens de acesso dos portais Unidas e Refran e o par de chaves VAPID do push estão escritos dentro das Edge Functions, não nos secrets do Supabase. Rotacioná-los hoje exige alterar código.

Achados de segurança no banco

● Quatro tabelas são legíveis sem login

`vehicles` (com placa, chassi e renavam), `daily_occupancy`, `supply_contracts` e `vexsoft_ingest` têm policy de SELECT para `anon`. Como a chave anônima está publicada no HTML do dashboard e dos portais, **qualquer pessoa com a URL consegue ler essas tabelas.**

O item de maior risco comercial da lista não é a placa: é `supply_contracts`, que expõe os termos dos dois lados do negócio em Salvador. Se a Refran vir o que a Unidas paga — ou o contrário — isso é problema de negociação, não de compliance.

Isso é o que permite o dashboard funcionar como HTML estático, então fechar exige mudar de onde ele lê.

Plano de migração (sem janela de indisponibilidade):

1. Criar `/api/data` no PWA — mesma casa do `/api/ask`, mesma autenticação por header `x-ff-key`, mesmo `service_role` que já está configurado. Devolve os quatro conjuntos que o

- `loadAll()` consome hoje: veículos, ocupação diária, contratos e preços. Nenhuma variável de ambiente nova.
2. Apontar o `loadAll()` do dashboard para essa rota, **mantendo a leitura direta como fallback** enquanto durar a transição.
 3. Confirmar que o painel carrega igual, com os mesmos números.
 4. Só então remover as policies `anon: Dashboard read access` em `vehicles`, `anon read occupancy`, `anon read contracts` e `anon read vexsoft_ingest`.
 5. Remover o fallback.

Os portais Unidas e Refran **não são afetados**: eles usam a chave anônima apenas como cabeçalho de gateway das Edge Functions (`verify_jwt`), não leem tabela direto.

Uma observação que aparece nesse levantamento: o dashboard já lê `service_pricing` sem ter policy para `anon` — a consulta volta vazia e ninguém percebe, porque os preços estão como constantes no JavaScript. Vale corrigir junto, passando a usar a tabela de verdade.

● Funções `SECURITY DEFINER` expostas — corrigido em 14/08

`launch_daily_charges()` era chamável por qualquer um com a chave anônima, o que permitia **disparar o lançamento de cobranças** de fora. `get_pending_users()` devolvia e-mails de `auth.users` para `anon`.

Ambas foram revogadas de `anon` (e `launch_daily_charges` também de `authenticated`) na migration `harden_definer_function_grants`. O cron continua funcionando porque roda como `postgres`.

● Oito funções sem `search_path` fixo

Funções `SECURITY DEFINER` legadas do ProLine sem `SET search_path` — vetor clássico de escalonamento. `ai_read_sql` e `launch_daily_charges` **estão corretas**.

● Views `clients` e `partners` são `SECURITY DEFINER`

Executam com as permissões de quem as criou, não de quem consulta. São legado do ProLine e não têm uso no pátio.

● Proteção contra senhas vazadas desligada

O Supabase Auth tem a checagem contra o HaveIBeenPwned desabilitada. Ligar em Auth → Password Protection.

● Chave anônima escrita dentro de uma função do banco

`notify_entrada_pendente()` guarda a chave anônima em texto claro no corpo da função — visível para quem consegue ler `pg_proc`, inclusive pelo assistente de IA.

Integridade de dados

● Não havia `UNIQUE` impedindo cobrança duplicada — resolvido em 14/08

A idempotência dependia inteiramente do `NOT EXISTS` dentro de `launch_daily_charges()`, e qualquer inserção fora dela podia duplicar cobrança sem o banco reclamar. **E duplicou:** três linhas ativas, incluindo uma desmobilização de R\$ 88 cobrada duas vezes (ver histórico).

Índice `uniq_charge_vehicle_service_date` criado sobre `(vehicle_id, service_key, charge_date)` apenas para linhas não canceladas. As duplicatas existentes foram canceladas antes.

Efeito colateral a conhecer: dois extras iguais no mesmo veículo e no mesmo dia (dois `remocao_plotagem` em 10/08, por exemplo) passam a ser recusados. Se acontecer de verdade, use datas diferentes ou uma única cobrança com o valor somado.

● O modelo não representa reentrada

`vehicles.plate` é UNIQUE global. Um carro que passa duas vezes pelo pátio não pode ter dois registros — a segunda passagem sobrescreve a primeira ou fica só anotada em `notes`. Caso conhecido: TVH8D65 / TYH8D65.

● Preços hardcoded na tela de pendências

`app/home/pendencias/actions.ts` escreve 190 / 123,50 / 10 / 6,50 e dois `pricing_id` fixos, enquanto `save.ts` lê tudo de `service_pricing`. Se a tabela de preços mudar, o fluxo de pendências passa a lançar valor errado silenciosamente.

● Pendências não são filtradas por pátio

Tanto o contador da home quanto a listagem trazem pendências de **todos** os pátios. Bira consegue ver e resolver uma pendência de Natal. `resolverEntrada` também não valida se o pátio bate.

● Sem transação no fluxo de pendências

O veículo é criado e as cobranças são inseridas em passos separados. Se o segundo falhar, sobra um veículo `finalizado` sem nenhuma cobrança.

● O ciclo de faturamento não fecha no banco

Das 3.527 cobranças, 2.363 estão `pending` e 1.164 `cancelled`. **Nenhuma** está `invoiced` ou `paid`. Os campos existem e não são usados: o faturamento real acontece fora do sistema.

Funcionalidades incompletas

● Push não chega a ninguém

`push_subscriptions` está **vazia**. Todo o caminho existe — opt-in, trigger, Edge Function, service worker — mas nenhum operador aceitou a notificação. No iOS, o botão só aparece com o app instalado na tela de início.

● Não existe UI de admin

Lançar extras, corrigir cobranças e ajustar datas ainda é trabalho de SQL. O assistente de IA reduziu bastante o atrito, mas ele é **somente leitura** por decisão de projeto.

● **Operador não tem como trocar a senha**

Não existe fluxo de "esqueci minha senha" nem de troca no primeiro acesso. O reset é manual, pelo painel do Supabase.

● **Vistorias antigas não são reprocessadas**

O pipeline só olha e-mails dos últimos 3 dias. Qualquer retroativo continua sendo trabalho manual.

● **Os portais estão duplicados**

`fleetflow-unidas/index.html` e `patio-pwa/public/unidas.html` são idênticos byte a byte (idem Refran). Duas cópias, dois deploys, uma chance de divergirem.

Pendências de operação

Item	Desde
Backfill de dez/2025 a mar/2026 — cerca de 30 veículos sem nenhuma cobrança	14/08
Natal: vistoria órfã <code>TES3H85</code> e varredura de tudo que passou por NAT de abril em diante (zero movimento desde março)	14/08
Recife: 10 desmobilizações com entrada por vistoria e sem data de saída, dependendo de conferência física	14/08
Recife: duas ativações 0 km (<code>TZX9A97</code> , <code>TZX9A94</code>) com vistoria de saída e sem data de entrada	14/08
Treinamento do Diego (REC) — quase não registra movimentos no PWA; foi o que produziu quase todo o passivo de conciliação	24/07
Datas a confirmar: <code>TDZ1B66</code> , <code>TEC5H72</code> , <code>TYG3I70</code> , <code>TZP9G94</code> , <code>TYG8G55</code>	11/08
Extras de adesivo não lançados: <code>UAA9C82</code> (R\$ 250) e <code>TEV9G06</code> (R\$ 150). As placas não existiam em maio, mas existem hoje — o bloqueio provavelmente caiu	30/05
TCM9A04 — apareceu na conferência física de REC em julho e não reaparece em nenhum registro posterior	24/07
CNPJ real da Unidas (hoje é placeholder)	24/07
Conferir a primeira fatura da Unidas no fechamento	24/07
Rotacionar chaves expostas em conversas: service role e Anthropic; e também token do Telegram e chave do Gemini	25/05
Desligar formalmente o bot de Telegram	25/05

Performance

Sem impacto hoje (o banco é pequeno), mas vale saber antes de crescer:

- **43 chaves estrangeiras sem índice**, incluindo `vehicle_history.vehicle_id`, `vehicles.client_id` e `daily_occupancy.client_id`
- **26 policies de RLS reavaliam `auth.uid()` por linha** — a correção é envolver em `(select auth.uid())`
- **19 conjuntos de policies permissivas duplicadas**, todas avaliadas em OR a cada consulta

`vehicle_service_charges` cresce cerca de 40 linhas por dia.

Limpeza sugerida

- `Cursor/` — pasta vazia
- Três ZIPs do ProLine somando ~616 MB, redundantes com a pasta `proline/`
- `setup-fleetflow.sh` — aponta para o repositório do ProLine, não deve ser executado
- `lib/supabase/admin.ts` — código morto, ninguém importa
- `supabase/migrations/20260525_create_desmobilize_vehicle_rpc.sql` — a função foi removida do banco, a migration ficou
- Arquivos de lock órfãos (`~$...docx`, `..~lock...#`)